
Image-based waste-bin collection decisions

Marco Cesarato
EPICODE Institute of Technology
Computer Vision
18 July 2026

Abstract

Waste collection teams lose time when trucks visit bins that still have space. This project tests whether one bin photo can provide a useful collection signal. The classifier predicts whether the visible bin has space or needs collection.

The repository contains a 5,654-image subset of the Outdoor Garbage Classification Dataset. The code maps its fill-level tags to **has_space** and **needs_collection**. A frozen, ImageNet-initialized ResNet50 backbone and a two-class softmax head reach 0.8788 accuracy and 0.8596 macro F1 after 5 epochs. An OpenCV HOG + Linear SVM comparison reaches 0.7053 accuracy and 0.6734 macro F1. Both results use the same 1,130-image evaluation split. Because this split is also supplied to training as validation data, the measurements are exploratory rather than final test results. A heuristic threshold policy labels 19.3% of predictions as **review**; the prototype does not implement a review queue. Limitations include the split design, class balance, uncalibrated softmax scores, and absence of non-bin examples.

1 Problem statement

1.1 Why the problem is relevant

Collection timing involves a practical trade-off. An early visit uses fuel and route time before the bin is full, while a late visit may leave waste overflowing. A photograph can give operators an extra signal when they decide which stops to prioritize.

This prototype evaluates that signal from a single image. Its two output classes are **has_space** and **needs_collection**.

1.2 Decision task and project scope

The intended use is limited to collection triage for local services. The prediction should support an operator, not make an autonomous scheduling decision. Inputs are expected to contain a waste bin; the dataset has no rejection class for unrelated images.

2 Methodology: data, labels, and architecture

2.1 Dataset and target labels

The repository contains 5,654 images from multiple public datasets. The local subset includes 3,768 images tagged **is_empty** and 1,886 tagged **is_full**. It also contains 69 secondary **is_scattered** tags: 57 occur with **is_full** and 12 with **is_empty**. The loader ignores **is_scattered** and models only the fill-level tag.

2.2 Label mapping

The dataset provides two fill-level tags:

- **is_full**: at least one visible bin is full or overflowing.
- **is_empty**: the bin still has free space. This can mean fully empty or partly filled.

The JSON labels map directly to the two target classes, with one qualification: **is_empty** means that the bin can still accept waste, not necessarily that it is completely empty. Table 1 gives the mapping used in the code and report.

Original tag	Final class
is_empty	has_space
is_full	needs_collection

Table 1: Label mapping.

The target does not model street conditions or non-bin images. Although **is_scattered** records some loose waste, that secondary label is outside the current two-class task.

2.3 Training and evaluation split

Within each class, the code sorts records by numeric image ID and assigns the highest-numbered 20% to the evaluation split. This preserves the class ratio and gives the same files on every run, but it is neither random nor group-aware. Table 2 lists the counts.

Evaluation images do not contribute gradient updates, but `model.fit` uses them as validation data. Model comparison and threshold analysis also use this split. The reported values are exploratory evaluation results rather than measurements from an untouched final test set.

Split	Images	has_space	needs_collection
Train	4,524	3,015	1,509
Evaluation	1,130	753	377

Table 2: Deterministic stratified train/evaluation split.

2.4 Dataset considerations

The dataset has about twice as many `has_space` images as `needs_collection` images. The evaluation split keeps a similar ratio.

The image order suggests repeated or related photo groups. A stronger experiment would reserve separate validation and test sets, then group the final split by bin, location, or photo session. That design would better measure performance on new scenes.

Because the data contains bin images only, the current system cannot reject non-bin uploads, unusual viewpoints, severe blur, night scenes, or blocked views. A real service would need labeled examples of those cases.

2.5 Image preprocessing

Preprocessing converts each uploaded image into the format expected by the model.

OpenCV reads each image, converts BGR to RGB, resizes it to 96 by 96 pixels, and scales pixel values to the range from 0 to 1. A layer inside the saved Keras model then applies the ResNet50 channel order and ImageNet mean subtraction. Training and API inference use the same path.

2.6 ResNet50 transfer-learning architecture

The classifier uses a ResNet50 backbone (He et al. 2016) initialized with ImageNet weights (Deng et al. 2009). The backbone remains frozen, while global average pooling, dropout at 0.3, and a two-unit softmax layer form the trainable head. This choice reuses general image features for a dataset of 5,654 images; no random-initialization ablation was performed.

Training runs for 5 epochs with batch size 32 and saves the result to `models/model.keras`. The API uses this CNN because it has the higher macro F1 of the two evaluated models.

The training split has 3,015 `has_space` images and 1,509 `needs_collection` images. Macro F1 gives both classes equal weight, while accuracy is influenced more

by the larger `has_space` class. Missing the smaller class is also the riskier operational error.

3 Experimental results

CNN metrics are computed in `src/models.py` with scikit-learn (Pedregosa et al. 2011). Accuracy measures the share of correct predictions. Precision, recall, and F1 are calculated for each class, and each macro score is the unweighted mean of the two class scores. The implementation passes both class labels explicitly and uses `zero_division=0`. Rows of the confusion matrix are ground-truth classes; columns are predicted classes.

3.1 CNN evaluation results

Table 3 reports the ResNet50 results on the evaluation split.

Metric	Value
Accuracy	0.8788
Macro Precision	0.8725
Macro Recall	0.8501
Macro F1	0.8596

Table 3: Scikit-learn metrics for the ResNet50 CNN on the evaluation split.

3.2 Comparison experiment

The HOG + Linear SVM model is a comparison experiment, not part of the API pipeline. It tests a hand-crafted feature representation against the ResNet50 features.

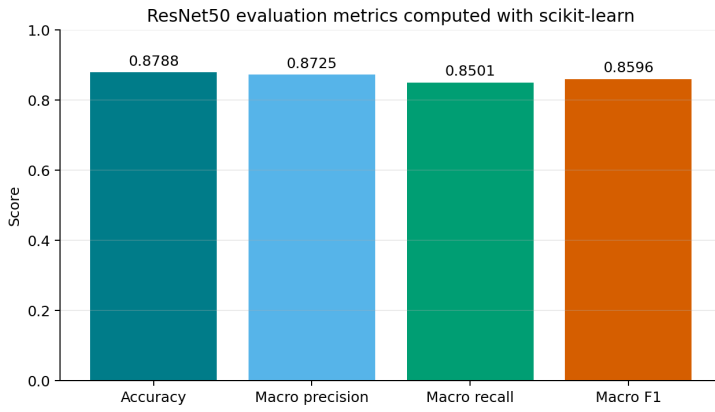
The source files are `src/compare/data.py`, which contains HOG extraction, split loading, and metric calculation, and `src/compare/models.py`, which runs the comparison evaluation command.

The comparison converts each image to grayscale, extracts a 64 by 64 Histogram of Oriented Gradients descriptor (Dalal and Triggs 2005), and trains a balanced `LinearSVC`. HOG describes local edge directions and shape, unlike the learned CNN representation. The values below were reproduced with OpenCV 4.13.0.92 and scikit-learn 1.8.0.

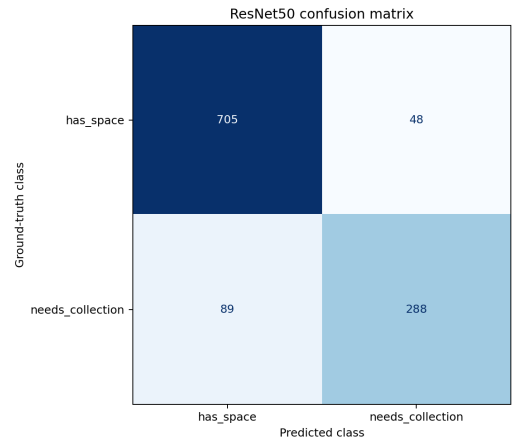
The HOG baseline follows the classical method shown in the course INRIA pedestrian-detection lab: grayscale images, OpenCV HOG descriptors, and a linear SVM classifier. The shared HOG layout uses 16 by 16 normalization blocks, 8 by 8 cells, an 8 by 8 block stride, and 9 orientation bins. These settings keep local edge detail, reduce lighting effects, and keep the feature vector small enough for a linear classifier.

For this dataset, the implementation reads the JSON labels, uses the shared train/evaluation split, and maps the fill-level tags to the two project classes. It also replaces the pedestrian-specific 64 by 128 window with a square 64 by 64 window and enables class balancing.

Table 4 compares evaluation performance for the API CNN and the HOG experiment. Macro F1 is used



(a) Accuracy and macro-averaged class metrics.



(b) Counts by ground-truth and predicted class.

Figure 1: Metric summary and confusion matrix for the saved CNN predictions on the evaluation split.

because the dataset is imbalanced; mostly predicting the larger class cannot produce a strong score for both classes.

Model	Input/features	Acc.	Macro Prec.	Macro Rec.	Macro F1
ResNet50 transfer CNN	RGB 96x96 pixels	0.8788	0.8725	0.8501	0.8596
Open CV HOG 64x64 + Linear SVM	Open CV HOG 64x64	0.7053	0.6713	0.6762	0.6734

Table 4: ResNet50 CNN and HOG + Linear SVM results on the same evaluation split.

The 5-epoch CNN makes 48 false collection alarms and misses 89 full-bin cases. These counts produce 0.8725 macro precision, 0.8501 macro recall, and 0.8596 macro F1.

On this split, the CNN exceeds the HOG comparison by 0.1862 macro F1 and reduces missed full-bin cases from 155 to 89. Figure 2 shows the difference. The API uses only the CNN.

Table 5 gives the CNN confusion matrix on the evaluation split.

	has_space	needs_collection
has_space	705	48
needs_collection	89	288

Table 5: CNN confusion matrix on the evaluation split.

The CNN correctly predicts 288 of 377 full bins and 705 of 753 bins with space.

A naive model that always predicts `has_space` would still reach 66.6% accuracy because that class is larger, but it would miss every bin that needs collection. Accuracy alone is not enough for this task. Macro F1 fits the task because it scores each class separately and then averages the two scores.

For the CNN, the confusion matrix gives an F1 of 0.9114 for `has_space` and 0.8079 for `needs_collection`. Their unweighted average gives the reported macro F1 of 0.8596.

3.3 Example predictions

Figure 3 contains four selected evaluation predictions from the saved CNN: one correct example from each

class, one error, and one case labeled `review`. The selection illustrates the available outputs and is not a random sample.

4 Postprocessing and decision routing

Postprocessing starts after class prediction. It converts the `needs_collection` softmax score into one of three routing labels and records how many cases remain outside `review`.

This step does not change the image or retrain the model. It only uses the output of the saved CNN.

4.1 Prediction outputs

The API loads `models/model.keras` once at startup. Its `confidence` field is the score for the predicted class, and the response also includes both class scores.

The policy reads the `needs_collection` score and returns `urgent`, `low`, or `review`. The scores have not been calibrated, so they are not presented as real-world probabilities.

4.2 Grad-CAM inspection

Grad-CAM uses gradients from a selected class score to weight convolutional feature maps from the ResNet backbone (Selvaraju et al. 2017). The resulting heatmap indicates which image regions most affected that score.

This provides a visual sanity check. Strong activation on background objects could indicate that the model relies on a shortcut rather than bin fill level.

Figure 4 shows one selected correct example from each class. In these two cases, the strongest activation overlaps the bin interior or rim. The generator prefers two fixed image IDs and otherwise chooses a correct image using clarity, size, and confidence. The figure is qualitative and does not establish behavior across the dataset.

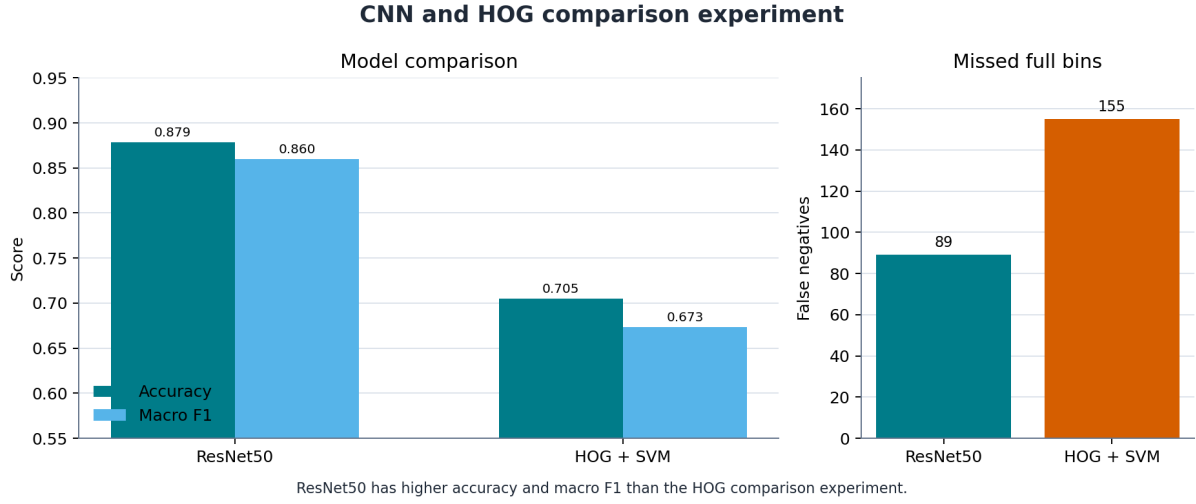


Figure 2: Model comparison between the ResNet50 CNN and the HOG + Linear SVM experiment.

4.3 Decision routing

The routing policy applies two fixed thresholds to the `needs_collection` softmax score. These thresholds are heuristic; they were not selected on an independent validation set. The API uses the same boundaries for `collection_priority` and `risk_level`:

- Scores of 0.70 or higher receive **urgent** priority and **high** risk.
- Scores of 0.15 or lower receive **low** priority and **low** risk.
- Scores between those boundaries receive **review** priority and **medium** risk.

If no score is available, the API returns **review** priority and **unknown** risk. The prototype does not provide a queue or reviewer workflow.

4.4 Routing effect on the evaluation split

The policy labels 218 of 1,130 predictions as **review**, leaving 912 non-review cases. Accuracy on that subset is 0.9298 at 80.7% coverage. This is selective accuracy, not an improvement to the classifier. Among 377 full-bin images, 267 are **urgent**, 68 are **review**, and 42 are incorrectly labeled **low**. The urgent route therefore captures 0.7082 of all full-bin images. If the denominator is limited to the 309 non-review full-bin cases, recall is 0.8641.

Table 6: Heuristic threshold policy on the CNN evaluation predictions.

Metric	Value
Urgent score threshold	0.70
Low-priority score threshold	0.15
Predictions labeled review	218
Review share	19.3%
Automatic decisions	912
Automatic coverage	80.7%
Automatic urgent decisions	289
Automatic low-priority decisions	623
Accuracy on automatic predictions	0.9298
Full bins labeled urgent	267
Full bins labeled review	68
Full bins still missed automatically	42
Urgent recall over all full bins	0.7082
Recall among non-review full bins	0.8641

Figure 5 summarizes the two thresholds and resulting evaluation counts.

The API response separates prediction details, routing output, and timing. `prediction` contains the label, predicted-class softmax score, and both class scores. `decision` contains the priority, risk level, and reason. `metadata` reports inference-path time.

5 API deployment

5.1 API and web interface

The web application in `src/app.py` loads the saved CNN model. FastAPI exposes the JSON prediction endpoint. Gradio supplies the browser upload page, which is mounted on the same FastAPI app at the application root.

Running Uvicorn from the project root starts both entry points. The Gradio page accepts a waste-bin image, shows a short prediction summary, and displays

Evaluation prediction examples



Figure 3: Selected evaluation predictions from the saved ResNet50 model. Each panel reports the ground-truth label, prediction, softmax score, and routing label.

the raw API-style JSON response. The API accepts requests at `POST /predict`.

The browser page reuses the prediction function called by the API. It receives an RGB NumPy array, converts it to OpenCV BGR, and calls the same pre-processing, inference, and routing code as `/predict`. Both interfaces therefore return the same label, scores, and routing fields.

The `/predict` endpoint accepts one uploaded file, decodes it with OpenCV, predicts the fill-level class, and applies the threshold policy. Table 7 lists the public response fields. The timer starts after decoding, so `processing_time_seconds` excludes upload transfer, file reading, image decoding, thread scheduling, and response serialization.

Table 7: Example API response fields for `POST /predict`.

Section	Field	Example value
prediction	label	needs_collection
	confidence	0.9812
prediction.scores	has_space	0.0188
	needs_collection	0.9812
decision	priority	urgent
	risk_level	high
decision	reason	Needs_collection risk 0.98 meets urgent threshold 0.70.
metadata	processing_time_seconds	0.1234

The deployment uses these choices:

- The model loads once when the application starts,

so requests do not reload it from disk.

- Image preparation, prediction, and routing run in a thread pool so the request handler does not block other asynchronous I/O.
- The endpoint accepts JPEG, PNG, WebP, and BMP uploads up to 10 MB.

The endpoint checks the declared media type, limits the bytes read, and returns a generic HTTP 500 message while logging the underlying exception. It still has no authentication or rate limit. The deployment is suitable for a local demonstration, not a public service.

6 Failure analysis: error cases and limits

The most important failure mode is a missed full bin: the CNN predicts `has_space` when the ground truth is `needs_collection`. This occurs 89 times on the evaluation split. After routing, 42 full-bin cases still receive the automatic `low` label. In practice, such an error could leave an overflowing bin uncollected.

False alarms are less severe but still costly. The CNN predicts `needs_collection` for 48 bins that actually

ResNet50 Grad-CAM examples

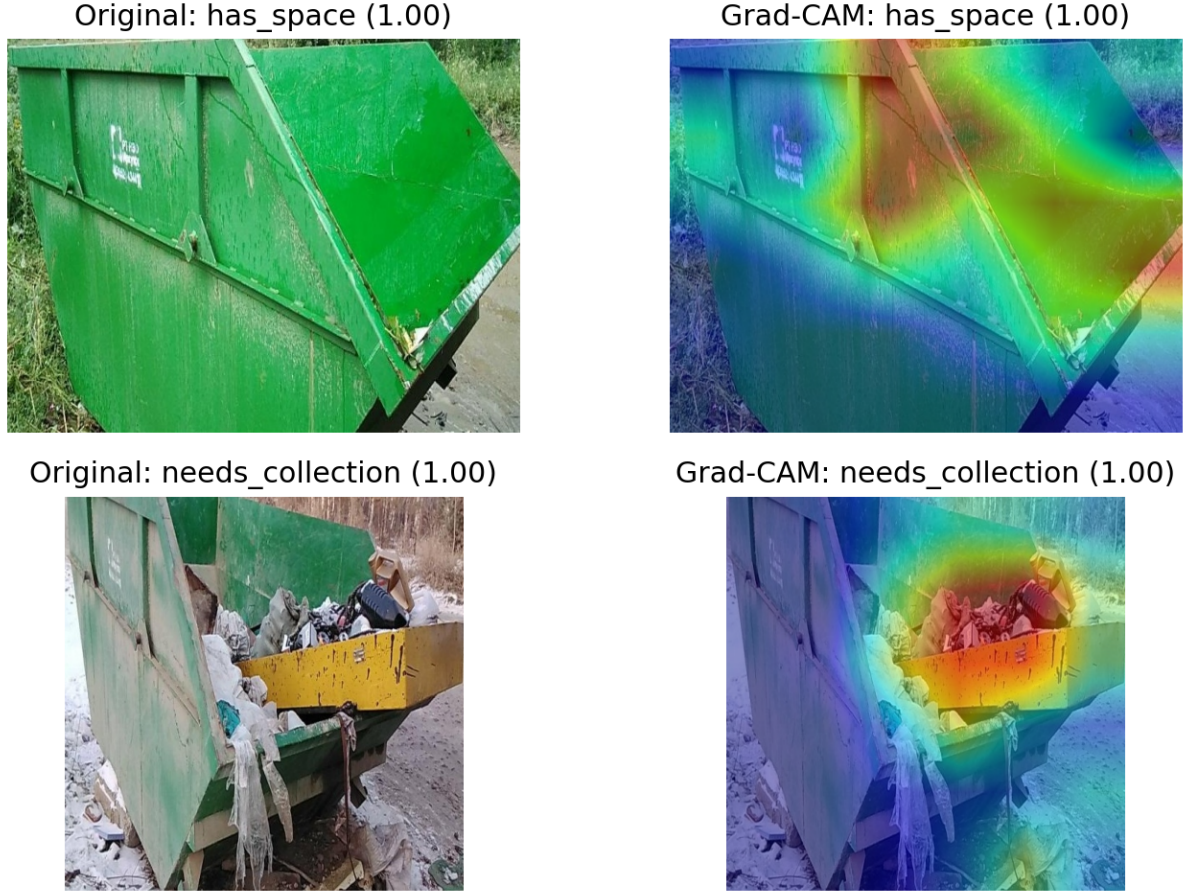


Figure 4: Selected correct ResNet50 predictions with Grad-CAM overlays. Warmer colors indicate greater contribution to the selected class score.

have space. This can send a truck to a bin that could have waited.

Possible causes include unclear fill levels, partly visible bins, bags near the rim, blocked views, lighting changes, camera angle, and background clutter. Because the deterministic split follows image ID order rather than group identity, related images may appear in both partitions. A group-based final test would better measure performance on new bins, locations, and photo sessions.

Only bin images are available for training. The model cannot reject unrelated uploads, severe blur, night scenes, blocked views, or unusual waste around the bin. A deployed service would need to reject or separately review those cases.

Further work should first define train, validation, and group-based test partitions. It should then add non-bin inputs and more difficult `needs_collection` examples.

7 Ethical considerations: bias, privacy, and human oversight

Dataset coverage creates a bias risk. If images come from only a few bins, streets, lighting conditions, seasons, or camera positions, the model may work better there than in other areas or with other bin types.

That gap could make service quality uneven. A service dataset should include varied locations, weather, bin designs, fill levels, and camera devices.

Privacy matters because street-level bin photos may accidentally include people, faces, house numbers, license plates, or private property. Images should be collected only when needed for the waste task. They should be stored for the shortest useful time and anonymized when possible. Public reports should avoid examples that show private details.

The system should support operators, not replace them. Human review is important for middle-band softmax scores, unusual images, and complaints. The model should not be used for fines, surveillance, or individual profiling because the dataset and labels support only a collection-priority task.

8 Conclusion

The prototype combines waste-bin image classification with threshold-based API routing. The saved model uses a frozen, ImageNet-initialized ResNet50 backbone and a two-class softmax head. On the 1,130-image evaluation split, it reaches 0.8788 accuracy and 0.8596 macro F1. The HOG + Linear SVM comparison reaches 0.7053 accuracy and 0.6734 macro F1, so the

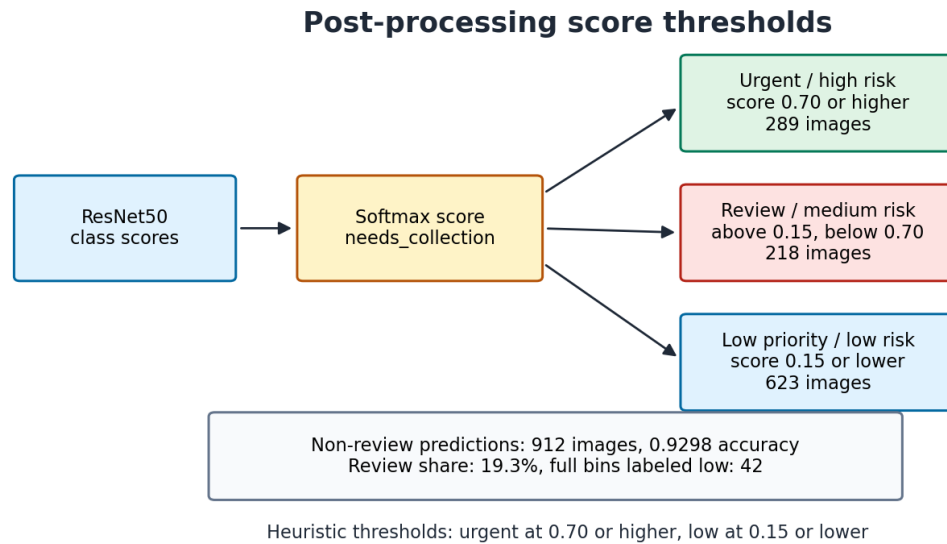


Figure 5: Heuristic routing of the ResNet50 `needs_collection` softmax score.

API uses the CNN.

The results need careful interpretation. The evaluation split contains 753 `has_space` and 377 `needs_collection` images, and training also uses it as validation data. The split follows numeric image ID and does not keep bins, locations, or photo sessions together. It is useful for comparing the two implementations in this project, but it is not an untouched final test.

The threshold policy operates on an uncalibrated `needs_collection` softmax score. It labels 289 predictions `urgent`, 623 `low`, and 218 `review`. Accuracy is 0.9298 on the 912 non-review cases, which corresponds to 80.7% coverage. Of 377 full-bin images, 267 are `urgent`, 68 are `reviewed`, and 42 are incorrectly labeled `low`. The prototype returns the review label but does not implement a review workflow.

Before operational use, the evaluation needs separate validation and group-based test sets. The data should also include unrelated uploads, difficult full-bin cases, and explicit privacy checks for street-level images. The API now limits uploads, validates declared media types, and hides internal exception details, but it still lacks authentication and rate limiting. It remains a local course prototype rather than a city service.

References

- He, Kaiming et al. (2016). “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- Deng, Jia et al. (2009). “ImageNet: A Large-Scale Hierarchical Image Database”. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255. DOI: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- Pedregosa, Fabian et al. (2011). “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12, pp. 2825–2830. URL: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Dalal, Navneet and Bill Triggs (2005). “Histograms of Oriented Gradients for Human Detection”. In: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 886–893. DOI: [10.1109/CVPR.2005.177](https://doi.org/10.1109/CVPR.2005.177).
- Selvaraju, Ramprasaath R. et al. (2017). “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization”. In: *2017 IEEE International Conference on Computer Vision*, pp. 618–626. DOI: [10.1109/ICCV.2017.74](https://doi.org/10.1109/ICCV.2017.74).